

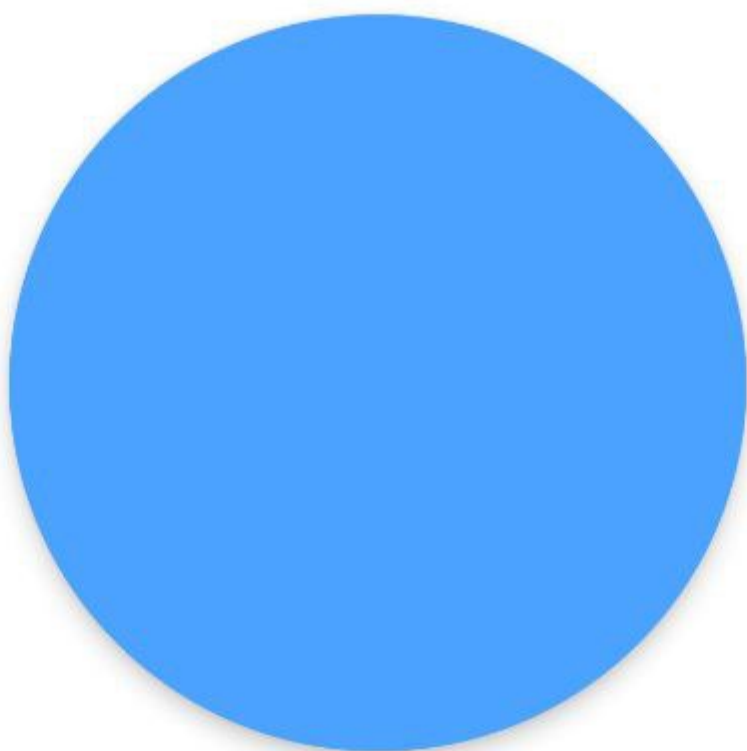
Lab 3

Тема: Використання жестів у React Native за допомогою react-native-gesture-handler.

Завдання: Розробити мобільний додаток – гру-клікер із використанням жестів, яка закріпить знання з теми взаємодії користувача з додатком у React Native.

Екрани

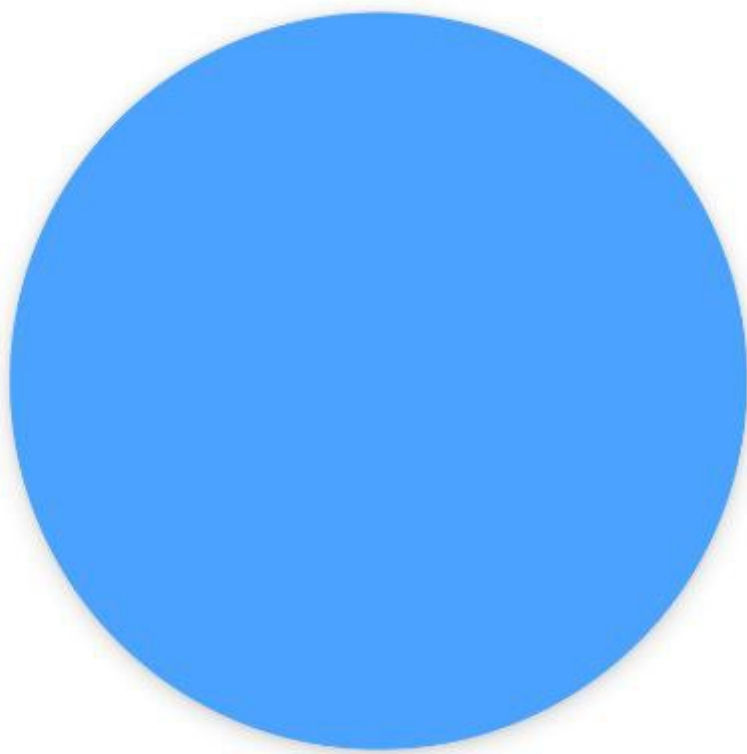
Очки: 68



Очки: 0



Очки: 68



Очки: 46



Подвійний клік ×5 5/5

Утримувати 3 с 1/1

Перетягнути об'єкт 1/1

Свайп вправо 1/1

Свайп вліво 1/1

Змінити розмір (pinch) 1/1

Отримати 100 очок 100/100

Подвійний клік ×5 5/5

Утримувати 3 с 0/1

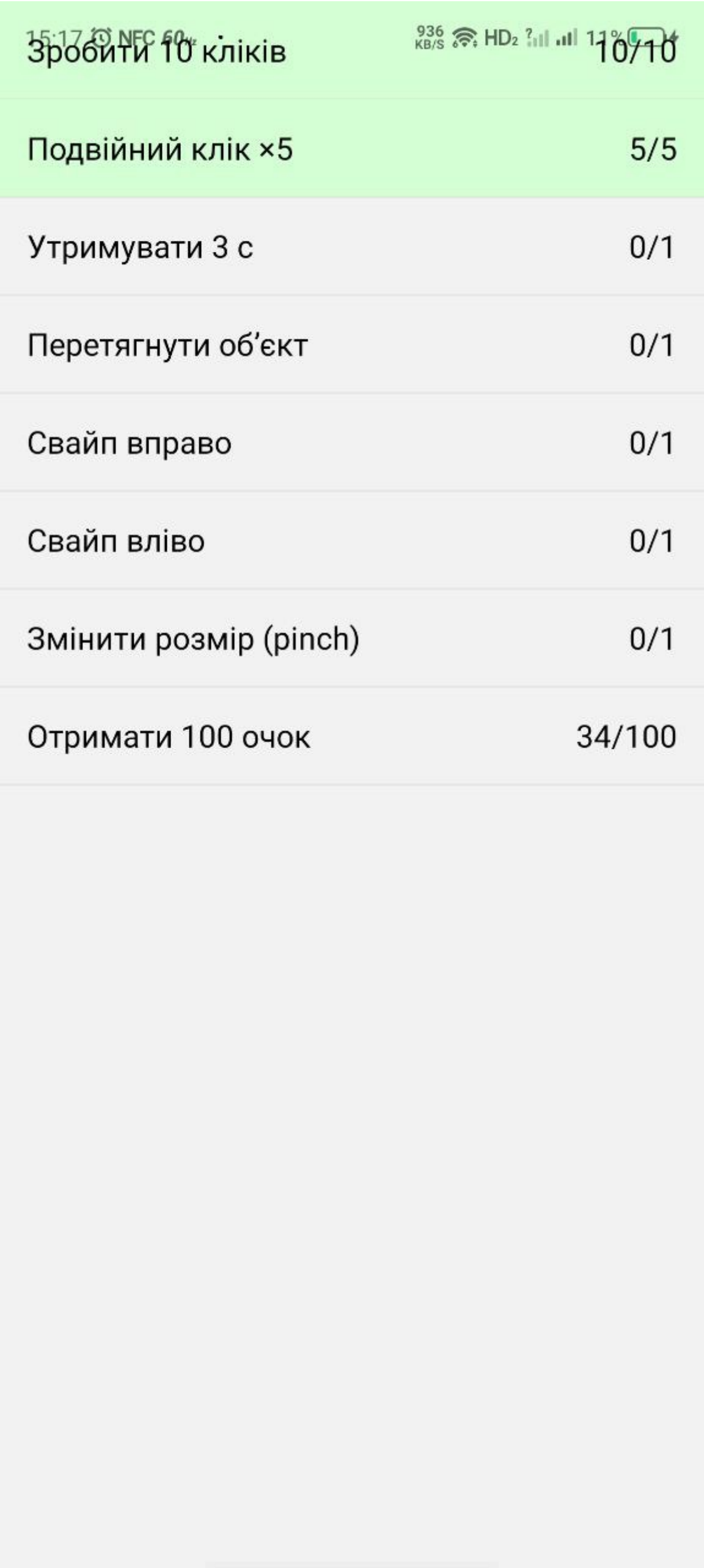
Перетягнути об'єкт 0/1

Свайп вправо 0/1

Свайп вліво 0/1

Змінити розмір (pinch) 0/1

Отримати 100 очок 24/100



App.js

```
/* Clicker Game
```

- Один екран – гра-клікер із жестами
- Другий екран – список завдань (FlatList)
- Навігація – React Navigation Stack

- Стан зберігається у React-Context

```
*/
```

```
import React, { useState, useRef, createContext, useContext } from 'react';
```

```
import {
```

```
  View,
```

```
  Text,
```

```
  StyleSheet,
```

```
  Animated,
```

```
} from 'react-native';
```

```
import {
```

```
  GestureHandlerRootView,
```

```
  TapGestureHandler,
```

```
  LongPressGestureHandler,
```

```
  PanGestureHandler,
```

```
  PinchGestureHandler,
```

```
  FlingGestureHandler,
```

```
  Directions,
```

```
  State, // ← потрібен для перевірки END
```

```
} from 'react-native-gesture-handler';
```

```
import { NavigationContainer } from '@react-navigation/native';
```

```
import { createStackNavigator } from '@react-navigation/stack';
```

```
import { Ionicons } from '@expo/vector-icons';
```

```
import { FlatList } from 'react-native-gesture-handler';
```

```
/*————— Context —————*/
```

```
const GameCtx = createContext(null);
```

```
const useGame = () => useContext(GameCtx);
```

```
/*————— Завдання —————*/
```

```
const makeTasks = () => ([
```

```
  { id: 'tap10', title: 'Зробити 10 кліків', goal: 10, progress: 0 },
```

```
  { id: 'dbl5', title: 'Подвійний клік ×5', goal: 5, progress: 0 },
```

```
  { id: 'hold3', title: 'Утримувати 3 с', goal: 1, progress: 0 },
```

```
  { id: 'drag', title: 'Перетягнути об'єкт', goal: 1, progress: 0 },
```

```
  { id: 'swpR', title: 'Свайп вправо', goal: 1, progress: 0 },
```

```
{ id: 'swpL', title: 'Свайп вліво', goal: 1, progress: 0 },
{ id: 'pinch', title: 'Змінити розмір (pinch)', goal: 1, progress: 0 },
{ id: 'pts100', title: 'Отримати 100 очок', goal: 100, progress: 0 },
]);

/*————— Гра —————*/

function GameScreen({ navigation }) {

  const { score, setScore, tasks, setTasks } = useGame();

  /* — Animated значення — */

  const pan = useRef(new Animated.ValueXY()).current;

  const scale = useRef(new Animated.Value(1)).current;

  const lastScale = useRef(1);

  /* — refs ієрархії жестів — */

  const singleTap = useRef();
  const doubleTap = useRef();
  const panRef = useRef();
  const pinchRef = useRef();
  const flingR = useRef();
  const flingL = useRef();
  const holdRef = useRef();

  /* — допоміжні — */

  const incTask = (id, d = 1) =>

    setTasks(ts => ts.map(t =>

      t.id === id ? { ...t, progress: Math.min(t.goal, t.progress + d) } : t));

  const addPts = (p) => {

    setScore(s => s + p);

    incTask('pts100', p);

  };

  /* — Gesture callbacks — */

  /* TAP */

  const onSingleTap = () => { addPts(1); incTask('tap10'); };
```

```
const onDoubleTap = () => { addPts(2); incTask('dbl5'); };

/* LONG PRESS 3 s → очки тільки після відпускання */

const onLongPress = ({ nativeEvent }) => {

  if (nativeEvent.state === State.END) {

    addPts(10);

    incTask('hold3');

  }

};

/* PAN */

const onPan = Animated.event(

  [{ nativeEvent: { translationX: pan.x, translationY: pan.y } }],

  { useNativeDriver: false },

);

const panEnd = ({ nativeEvent }) => {

  if (nativeEvent.state === State.END) {

    pan.extractOffset();

    incTask('drag');

  }

};

/* PINCH */

const onPinch = Animated.event(

  [{ nativeEvent: { scale } }],

  { useNativeDriver: false },

);

const pinchEnd = ({ nativeEvent }) => {

  if (nativeEvent.state === State.END) {

    lastScale.current *= nativeEvent.scale;

    scale.setValue(lastScale.current);

    incTask('pinch');

  }

};

/* FLING →/← (очки лише в State.END) */
```

```
const fling = dir => ({ nativeEvent }) => {

  if (nativeEvent.state === State.END) {

    addPts(Math.floor(Math.random() * 11) + 5);    // 5-15

    incTask(dir === 'R' ? 'swpR' : 'swpL');

  }

};

/* — UI — */

return (

  <GestureHandlerRootView style={{ flex: 1 }}>

    <View style={styles.container}>

      <View style={styles.header}>

        <Text style={styles.score}>Очки: {score}</Text>

        <Icons name="list" size={28} onPress={() => navigation.navigate('Tasks')} />

      </View>

      {/* Ієрархія всіх жестів */}

      <FlingGestureHandler

        ref={flingR} direction={Directions.RIGHT}

        simultaneousHandlers={[flingL, panRef, pinchRef]}

        onHandlerStateChange={fling('R')}>

        <FlingGestureHandler

          ref={flingL} direction={Directions.LEFT}

          simultaneousHandlers={[flingR, panRef, pinchRef]}

          onHandlerStateChange={fling('L')}>

          <PanGestureHandler

            ref={panRef} avgTouches

            onGestureEvent={onPan} onHandlerStateChange={panEnd}

            simultaneousHandlers={pinchRef}>

            <PinchGestureHandler

              ref={pinchRef}

              onGestureEvent={onPinch} onHandlerStateChange={pinchEnd}

              simultaneousHandlers={panRef}>
```

```
<LongPressGestureHandler

  ref={holdRef} minDurationMs={3000}

  onHandlerStateChange={onLongPress}>


<TapGestureHandler

  ref={doubleTap} numberOfTaps={2}

  onActivated={onDoubleTap}>


<TapGestureHandler

  ref={singleTap} waitFor={doubleTap}

  onActivated={onSingleTap}>


<Animated.View style={[

  styles.bubble,

  {

    transform: [

      { translateX: pan.x },

      { translateY: pan.y },

      { scale },

    ],

    },

  ]}/>

</TapGestureHandler>


</TapGestureHandler>

</LongPressGestureHandler>

</PinchGestureHandler>

</PanGestureHandler>

</FlingGestureHandler>

</FlingGestureHandler>

</View>

</GestureHandlerRootView>

);

}

/*————— Сторінка завдань —————*/
```

```
function TasksScreen() {

  const { tasks } = useGame();

  const renderItem = ({ item }) => (

    <View style={[styles.taskRow, item.progress >= item.goal && styles.done]}>

      <Text style={styles.taskText}>{item.title}</Text>

      <Text style={styles.taskText}>{item.progress}/{item.goal}</Text>

    </View>

  );

  return <FlatList data={tasks} renderItem={renderItem} keyExtractor={i => i.id} />;
}

/*————— Навігація + Provider —————*/

const Stack = createStackNavigator();

export default function App() {

  const [score, setScore] = useState(0);

  const [tasks, setTasks] = useState(makeTasks());

  return (

    <GameCtx.Provider value={{ score, setScore, tasks, setTasks }}>

      <NavigationContainer>

        <Stack.Navigator screenOptions={{ headerShown: false }}>

          <Stack.Screen name="Game" component={GameScreen} />

          <Stack.Screen name="Tasks" component={TasksScreen} />

        </Stack.Navigator>

      </NavigationContainer>

    </GameCtx.Provider>

  );
}

/*————— Стили —————*/

const styles = StyleSheet.create({

  container: { flex: 1, backgroundColor: '#fff', justifyContent: 'center', alignItems: 'center' },

  header: { position: 'absolute', top: 40, left: 20, right: 20,

    flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },

  score: { fontSize: 24, fontWeight: '600' },

  bubble: { width: 120, height: 120, borderRadius: 60, backgroundColor: '#4ba3ff', elevation: 5 },
```



```
taskRow: { flexDirection: 'row', justifyContent: 'space-between',  
           padding: 14, borderBottomWidth: StyleSheet.hairlineWidth, borderColor: '#ccc' },  
taskText: { fontSize: 16 },  
done:     { backgroundColor: '#d4ffd4' },  
});
```